System Analysis & Design

DEPI · System Analysis & Design · UK Railway Journeys Analysis: Revenue, Delays & Refunds Dashboard

9	10	7	8	5
Use cases	Functional requirements	Non-functional requirements	Tables in the star schema	ETL stages

Project information

Project title	UK Railway Journeys Analysis: Revenue, Delays & Refunds Dashboard	**Program**	DEPI, Digital Egypt Pioneers Initiative
Team	OMNI	**Track**	Data Analysis
Team leader	Nada Ahmed El-Sayed	**Instructor**	Ahmed Ayman
Phase	System Analysis & Design (Oct 25 – Nov 1, 2026)	**Deadline**	6 November 2026

1. Problem Statement & Objectives

1.1 Problem statement

About **13%** of the journeys in the dataset were delayed (7.2%) or cancelled (5.9%), and **1,118** passengers requested a refund. The cause of a disruption is recorded inconsistently, and there is no ready view showing where, when, and why journeys fail. Without this view, management cannot decide which routes, stations, and causes to fix first, or understand how disruptions affect revenue and refunds.

1.2 Objectives

#	Objective
1	Clean and standardize the data and document every step.
2	Build a reproducible ETL pipeline using Python, Power Query (M Language), Databricks (SQL), and Power BI.
3	Design a Star Schema for the data.
4	Define the KPIs: Total Revenue, On-Time %, Delay %, Cancellation %, Average Delay Minutes, and Refund Rate.
5	Deliver an interactive Power BI dashboard that answers the business questions.

System Analysis & Design

#	Objective
6	Turn the findings into clear, data-backed recommendations.
7	(Optional stretch goal) Build a baseline machine learning model for disruption risk, only if time allows.

1.3 Business questions

#	Business question
Q1	Which routes and stations have the most delays and cancellations?
Q2	What are the main causes of delays and cancellations, and how do they change over time?
Q3	How much revenue is linked to disrupted journeys and refund requests?
Q4	How do ticket type, ticket class, and railcard affect revenue?
Q5	When are the peak purchase and travel times?
Q6	Do online and station purchases behave differently?

2. System Actors & Use Cases

This project is an analytics system, so the users are the people who read the dashboard, plus the team members who prepare the data.

2.1 Actors

Actor	Description
Operations Manager	Main dashboard user. Decides which routes, stations, and causes to fix first.
Revenue / Commercial Analyst	Reviews revenue by ticket type, class, railcard, and purchase channel.
Customer Service Lead	Reviews refund requests and their link to disruptions.
Data Team (OMNI)	Cleans, transforms, models, and validates the data before it reaches the dashboard.

2.2 Use cases

System Analysis & Design

ID	Use case	Actor	Description
UC1	View KPI overview	Operations Manager	Sees Total Revenue, On-Time %, Delay %, Cancellation %, Average Delay Minutes, and Refund Rate on one page.
UC2	Analyze delays by route and station	Operations Manager	Finds the routes and stations with the most delays and cancellations (Q1).
UC3	Analyze delay causes over time	Operations Manager	Compares causes of disruption by month and week (Q2).
UC4	Analyze revenue impact of disruptions	Revenue Analyst	Sees revenue linked to disrupted journeys and refund requests (Q3).
UC5	Compare ticket type, class, and railcard	Revenue Analyst	Sees how each affects revenue (Q4).
UC6	Find peak purchase and travel times	Operations Manager	Sees demand by hour, weekday, and month (Q5).
UC7	Compare online and station purchases	Revenue Analyst	Compares behavior across purchase channels (Q6).
UC8	Filter and drill down	All dashboard users	Uses slicers (date, station, ticket type, status) to filter all pages.
UC9	Prepare and validate data	Data Team	Cleans, transforms, loads, and checks the data before publishing.

3. Requirements

3.1 Functional requirements

ID	Requirement	Linked to
FR1	The dashboard shall display the six core KPIs: Total Revenue, On-Time %, Delay %, Cancellation %, Average Delay Minutes, and Refund Rate.	UC1
FR2	The dashboard shall show delays and cancellations by route and by station.	UC2, Q1
FR3	The dashboard shall show standardized delay causes and their trend over time.	UC3, Q2
FR4	The dashboard shall show revenue linked to disrupted journeys and refund requests.	UC4, Q3
FR5	The dashboard shall compare revenue by ticket type, ticket class, and railcard.	UC5, Q4
FR6	The dashboard shall show purchase and travel patterns by hour, weekday, and month.	UC6, Q5
FR7	The dashboard shall compare online and station purchases.	UC7, Q6

ID	Requirement	Linked to
FR8	All pages shall support slicers (date, station, ticket type, journey status) and cross-filtering.	UC8
FR9	The data pipeline shall standardize inconsistent values (for example, delay reasons) and record every cleaning rule.	UC9
FR10	Delay duration shall be calculated correctly for journeys that arrive after midnight (using the arrival date as well as the time).	UC9

3.2 Non-functional requirements

ID	Category	Requirement
NFR1	Performance	Dashboard pages should load and respond to slicers in a few seconds on the full 31,653-row dataset.
NFR2	Accuracy	Each KPI is defined once in a shared document and calculated the same way everywhere (SQL and DAX).
NFR3	Reproducibility	The pipeline can be re-run from `railway.csv` and produce the same results.
NFR4	Data integrity	Blank Actual Arrival Time (cancelled journeys) and blank Reason for Delay (on-time journeys) are expected and must not be deleted or filled. The Railcard value `None` must be kept as a real category.
NFR5	Usability	Consistent layout, colors, and labels; each page answers a clear business question.
NFR6	Maintainability	Code is commented and the repository is organized by project stage.
NFR7	Security	The dataset contains no passenger names or personal identifiers. The repository holds only the project data.

3.3 Constraints and limitations

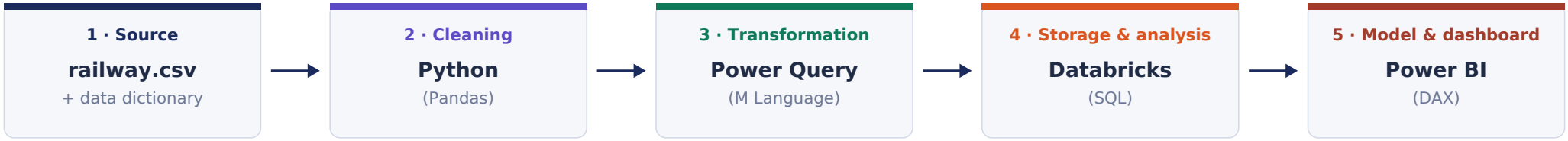
- The data covers only four months (1 Jan to 30 Apr 2024), so seasonal trends cannot be concluded.
- Actual refund amounts are not available in the data.
- Real-time data and production deployment of the ML model are out of scope.

System Analysis & Design

DEPI · System Analysis & Design · UK Railway Journeys Analysis: Revenue, Delays & Refunds Dashboard

4. Software Architecture

The system is a batch analytics pipeline. Data moves in one direction, from the raw file to the dashboard. **Architecture style:** layered ETL pipeline, ending in a star-schema semantic model that feeds the dashboard.



Across all stages: Figma (wireframes and layout) · GitHub (code and documentation)

Layer	Component	Role
1. Source	`railway.csv` + data dictionary	31,653 ticket records, 18 columns
2. Cleaning	Python (Pandas)	Standardize values, fix data types, validate
3. Transformation	Power Query (M Language)	Prepare data types, date and time fields
4. Storage & analysis	Databricks (SQL)	Load tables, validate, answer business questions
5. Modeling & presentation	Power BI (data model, DAX, dashboard)	Star Schema, KPIs, interactive visuals
6. Design	Figma	Wireframes and layout
7. Version control	GitHub	Code and documentation

5. Database Design & Data Modeling

5.1 Source columns (`railway.csv`)

Team check: the list below follows the standard column names of this dataset. Verify each name and type against `railway_data_dictionary.csv` before finalizing.

System Analysis & Design

#	Column	Group	Used for
1	Transaction ID	Purchase	Unique key
2	Date of Purchase	Purchase	Purchase date
3	Time of Purchase	Purchase	Purchase hour
4	Purchase Type	Purchase	Online vs Station (Q6)
5	Payment Method	Purchase	Payment analysis
6	Railcard	Ticket	Railcard analysis (Q4)
7	Ticket Class	Ticket	Class analysis (Q4)
8	Ticket Type	Ticket	Type analysis (Q4)
9	Price	Ticket	Revenue
10	Departure Station	Journey	Route and station analysis (Q1)
11	Arrival Destination	Journey	Route and station analysis (Q1)
12	Date of Journey	Journey	Travel date
13	Departure Time	Journey	Travel hour (Q5)
14	Arrival Time	Journey	Scheduled arrival
15	Actual Arrival Time	Performance	Delay minutes
16	Journey Status	Performance	On Time / Delayed / Cancelled
17	Reason for Delay	Performance	Delay causes (Q2)
18	Refund Request	Performance	Refund Rate (Q3)

5.2 Grain

One row in the fact table = one ticket transaction (one Transaction ID). The data check found no duplicate rows and no duplicate Transaction IDs.

System Analysis & Design

5.3 Star Schema (logical model)

One fact table surrounded by seven dimension tables.



Fact_Journeys

Column	Type	Key	Description
Transaction_ID	Text	PK	Unique ticket transaction
Purchase_Date_Key	Integer	FK	To Dim_Date (purchase date)
Journey_Date_Key	Integer	FK	To Dim_Date (journey date)
Departure_Station_Key	Integer	FK	To Dim_Station (departure)
Arrival_Station_Key	Integer	FK	To Dim_Station (arrival)
Ticket_Key	Integer	FK	To Dim_Ticket

Column	Type	Key	Description
Railcard_Key	Integer	FK	To Dim_Railcard
Purchase_Key	Integer	FK	To Dim_Purchase
Status_Key	Integer	FK	To Dim_Journey_Status
Delay_Reason_Key	Integer	FK	To Dim_Delay_Reason
Price	Decimal	Measure	Ticket price (revenue)
Delay_Minutes	Integer	Measure	Actual arrival minus scheduled arrival (derived)
Refund_Flag	Integer (0/1)	Measure	1 if a refund was requested
Purchase_Hour	Integer	Attribute	Hour extracted from Time of Purchase
Departure_Hour	Integer	Attribute	Hour extracted from Departure Time

Dimension tables

Table	Key	Attributes	Answers
Dim_Date	Date_Key	Date, Day, Weekday, Week, Month, Year	Q2, Q5
Dim_Station	Station_Key	Station name	Q1
Dim_Ticket	Ticket_Key	Ticket Type, Ticket Class	Q4
Dim_Railcard	Railcard_Key	Railcard (including `None`)	Q4
Dim_Purchase	Purchase_Key	Purchase Type, Payment Method	Q6
Dim_Journey_Status	Status_Key	Journey Status	Q1, Q3
Dim_Delay_Reason	Delay_Reason_Key	Standardized Reason for Delay	Q2

5.4 Design decisions

Decision	Reason
Dim_Date is used twice (purchase date and journey date)	One date table with two relationships: one active, the other activated in DAX when needed.
Dim_Station is used twice (departure and arrival)	Same idea. A route is the combination of departure and arrival.
Delay_Minutes is calculated using the arrival date and the time	Prevents wrong values for journeys that arrive after midnight.
Delay reasons are standardized (for example, Signal Failure and Signal failure become one value)	Prevents splitting totals across spellings.
Blank delay reasons and blank actual arrival times stay blank in the source data	They carry meaning and must not be filled. The team must still decide how Dim_Delay_Reason represents on-time journeys so relationships stay valid.

5.5 Physical schema and normalization

The dimension tables are small and stored once, so each descriptive value (such as a station name) is held in one place. The fact table keeps only keys and measures. The star schema is intentionally denormalized for fast reporting in Power BI. Primary keys are surrogate integers for the dimensions and the original Transaction ID for the fact table.

6. Data Flow & System Behavior

6.1 DFD Level 0 (context)

External entity	Data in	Data out
Data source (`railway.csv` + dictionary)	Raw ticket records	n/a
OMNI Analytics System	Raw records	Cleaned data, KPIs, dashboard
Dashboard users	Filter selections	KPIs, charts, insights

6.2 DFD Level 1 (processes)

Process	Input	Output	Tool
P1 Clean data	`railway.csv`	Cleaned dataset + cleaning rules file	Python (Pandas)
P2 Transform data	Cleaned dataset	Typed, structured tables	Power Query (M)

System Analysis & Design

Process	Input	Output	Tool
P3 Load and analyze	Transformed tables	SQL tables and analysis results	Databricks (SQL)
P4 Model and visualize	SQL tables	Star schema, DAX measures, dashboard	Power BI
P5 Validate	Dashboard values + SQL results	Test results	Team review

6.3 Workflow (activity flow)

Start → clean data → transform data → load into Databricks → run SQL analysis → build the star schema in Power BI → create DAX measures → build dashboard pages → validate against SQL results → publish and document.

6.4 ETL Pipeline (Extract, Transform, Load)

The ETL pipeline turns `railway.csv` into a clean star schema in five stages. Each stage has one tool, one output, and one check, so any result in the dashboard can be traced back to the raw file.

#	Stage	Tool	Owner	What happens	Output	Check
1	Extract	Python (Pandas)	Moustafa Elsayed	Read `railway.csv` and the data dictionary. Keep the Railcard value `None` as text, not Null. Read date and time columns as text first.	Raw dataframe (31,653 rows, 18 columns)	Row and column counts match the source file
2	Clean	Python (Pandas)	Moustafa Elsayed	Standardize delay reasons, check for duplicates, keep expected blanks as they are.	Cleaned dataset + cleaning rules file	No duplicate rows or Transaction IDs; still 31,653 rows
3	Transform	Power Query (M Language)	Omar Saad	Convert text columns to date, time, and decimal types. Build date and time fields. Calculate delay minutes using the arrival date and the time.	Typed, structured tables	Data types are correct; delay minutes checked on sample journeys, including after-midnight arrivals
4	Load & analyze	Databricks (SQL)	Omar Saad (database), Nada Ahmed (analysis)	Load the tables, validate them, and run the analysis queries for the business questions.	SQL tables + validation results	Row counts and total Price match the previous stage
5	Model & report	Power BI (DAX)	Dashboard team	Build the star schema (Fact_Journeys + 7 dimensions), DAX measures, and dashboard pages.	`.pbix` dashboard	Every KPI matches the SQL result

System Analysis & Design

Cleaning rules

These rules come from the data review in the Risk Assessment. Any new rule found during cleaning is added to the cleaning rules file.

Rule	Reason
Standardize delay reasons written in different ways (for example, Signal Failure and Signal failure; Weather and Weather Conditions).	Prevents splitting one cause into several totals (Q2).
Convert date and time columns from text to proper types in Power Query.	Needed for time analysis and delay calculation (Q5).
Calculate delay using the arrival date as well as the time.	Some journeys arrive after midnight; time alone gives wrong delay minutes.
Keep the Railcard value `None`.	It means "no railcard", not a missing value (Q4).
Keep blank Actual Arrival Time (cancelled journeys) and blank Reason for Delay (on-time journeys).	The blanks are expected and carry meaning; they are not deleted or filled.

Reconciliation checkpoints

After every stage the team confirms the same numbers, so no rows or values are lost silently:

- Total rows = 31,653.
- Sum of Price is the same as in the previous stage.
- Refund requests = 1,118.
- Journey Status counts (on time, delayed, cancelled) are the same as in the previous stage.

Reproducibility

The raw file is never edited. Each stage reads only the output of the previous stage, and the Python cleaning script, the Power Query steps, and the SQL scripts are stored in the GitHub repository, so the whole pipeline can be re-run from `railway.csv`.

7. UI/UX Design & Prototyping

7.1 Wireframes and mockups

Prepared in Figma by the UI/UX owner. Planned dashboard pages, one per group of business questions:

Page	Purpose	Questions
Overview	Six core KPIs and the main trend	KPIs

System Analysis & Design

Page	Purpose	Questions
Performance	Routes, stations, delays, cancellations	Q1
Causes	Delay causes and their change over time	Q2
Revenue & Refunds	Revenue linked to disruption and refunds	Q3, Q4
Demand & Channels	Peak times and online vs station	Q5, Q6

7.2 UI/UX guidelines

Area	Guideline
Layout	KPI cards on top, main chart in the center, filters in a consistent place on every page.
Colors	A limited palette. One color for "good" (on time) and one for "bad" (delayed, cancelled), used the same way on every page. Final palette to be filled in from the Figma file.
Typography	One font family, clear size hierarchy for titles, KPIs, and labels. Final fonts to be filled in from the Figma file.
Charts	Chart type chosen by the question: trends as lines, comparisons as bars, shares only when few categories.
Accessibility	High contrast, not relying on color alone, readable font sizes, clear titles on every visual.

8. Technology Stack & Integration

Area	Technology
Data cleaning	Python (Pandas)
Transformation	Power Query (M Language)
Storage and analysis	Databricks (SQL)
Modeling and dashboard	Power BI (DAX)
Design	Figma
Version control	GitHub

System Analysis & Design

Component diagram (summary): Python scripts, Power Query queries, Databricks tables, and the Power BI model depend on each other in one direction, left to right. Each component only reads the output of the previous one.

9. Items from the guidelines that do not apply

The DEPI guidelines are written for general software projects. For this analytics project, the following are not applicable. This table should be confirmed with the instructor.

Item	Status	Reason
API Documentation	Not applicable	The system has no API.
Deployment Diagram	Not applicable	There is no server or application deployment. The dashboard is a Power BI file.
Sequence Diagram	Not applicable	There are no interacting software components with messages between them.
State Diagram	Not applicable	There is no object with changing states.
Class Diagram	Not applicable	The system is not object-oriented software. The star schema replaces it.
Deployment Strategy	Not applicable	No hosting or scaling needed.

10. Remaining work before the 6 November deadline

- Verify the 18 column names and types against the data dictionary.
- Add the Use Case diagram and the Figma wireframe screenshots; fill the final colors and fonts.
- Confirm how Dim_Delay_Reason represents on-time journeys.
- Confirm the "not applicable" items with the instructor.
- Upload the final file to `3_System_Analysis_Design` on GitHub.